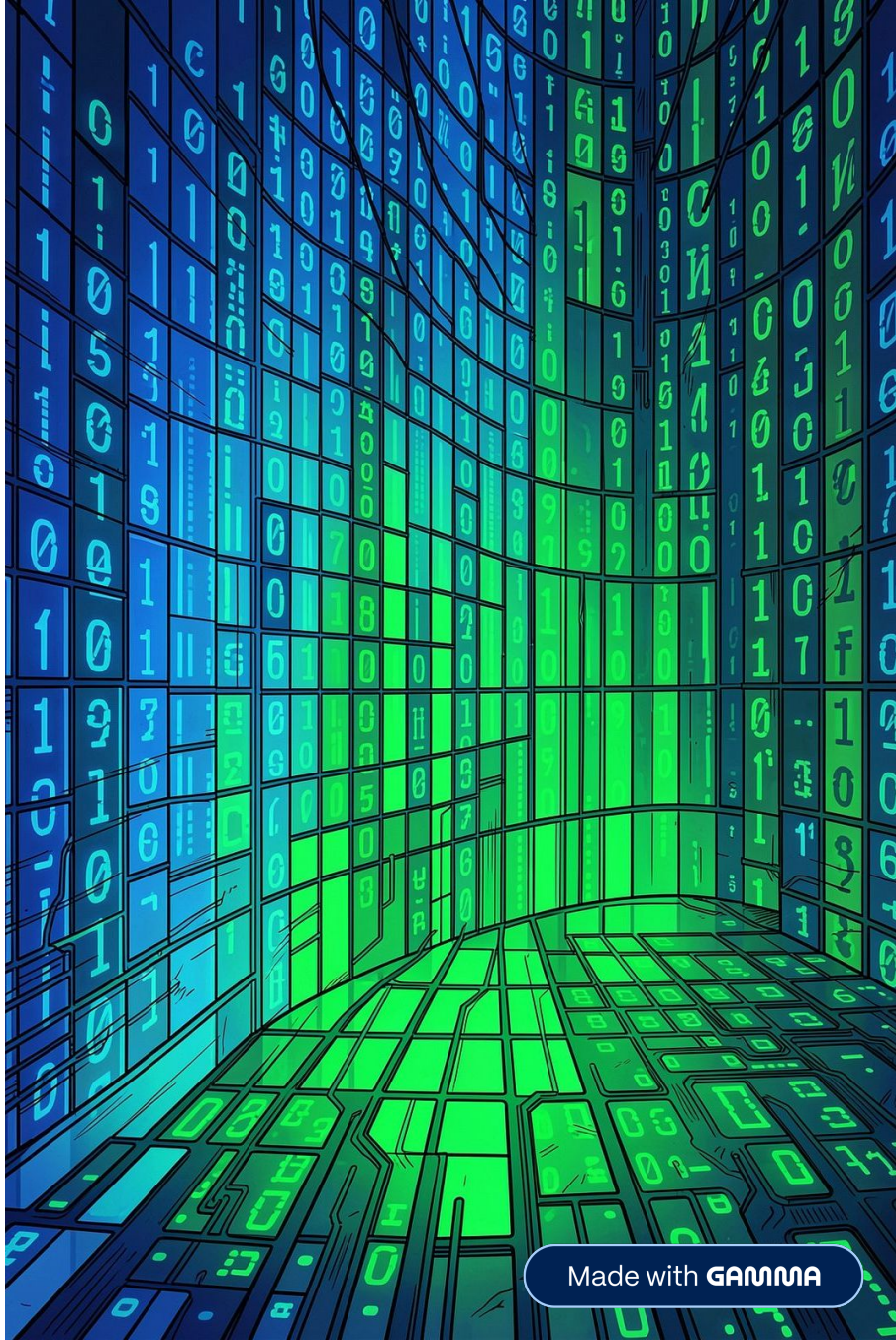
VISIÓN POR COMPUTADORA CON PYTHON

Imágenes Digitales y Primeros Pasos con OpenCV

Módulo 2 — Desde los fundamentos matemáticos de una imagen digital hasta la ejecución de un programa Python que carga, transforma, analiza, recorta y guarda imágenes con OpenCV y NumPy.

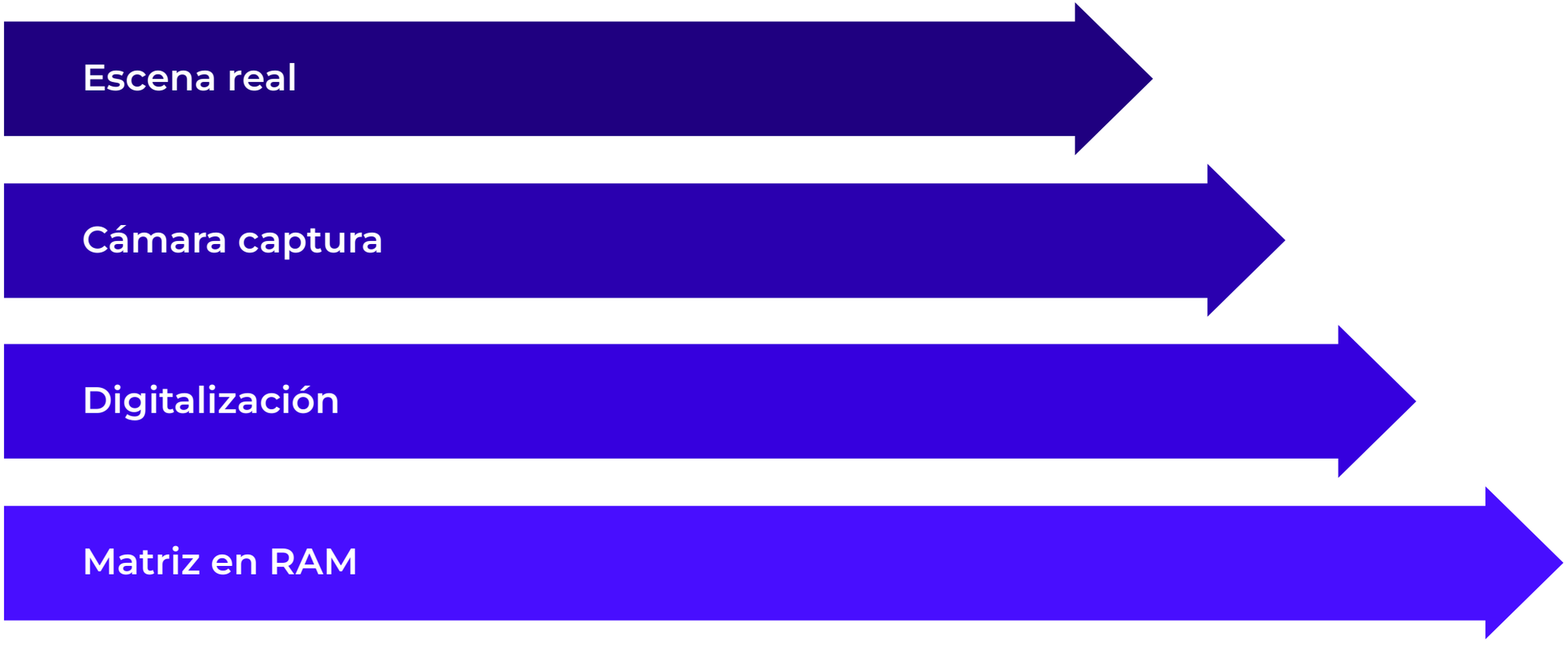


Made with GAMMA

BLOQUE 1 · INTRODUCCIÓN

¿Qué es una imagen digital?

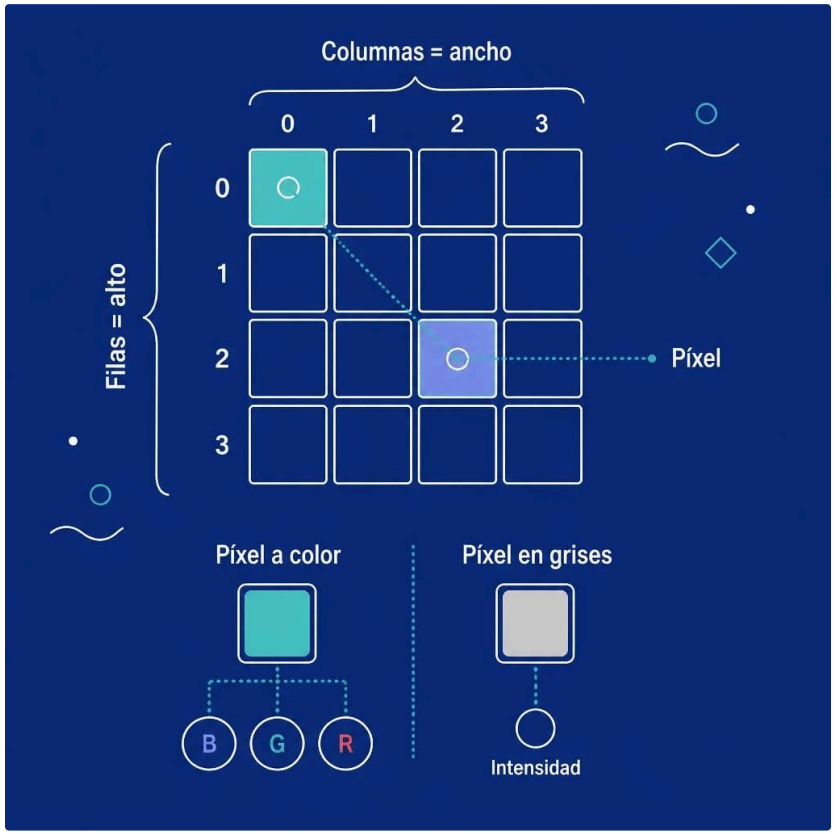
Una imagen digital es una **representación discreta de una escena**, organizada como una matriz de píxeles. Cada píxel almacena un valor numérico que representa su intensidad o color. El proceso de digitalización convierte la luz capturada por un sensor en información que Python puede procesar.



La diferencia clave con una imagen analógica es la **discretización**: la realidad continua se convierte en valores enteros organizados en filas y columnas.

Made with GAMMA

¿Cómo representa Python una imagen?



Una matriz de valores

- Píxeles: la unidad mínima, con filas y columnas
- Alto = número de filas;
- Ancho = número de columnas
- OpenCV guarda las imágenes a color en BGR (3 canales)
- Escala de grises: un solo valor de intensidad por píxel

Made with GAMMA

BLOQUE 1 · PÍXELES Y MATRICES

El píxel y la imagen como matriz

Un **píxel** (picture element) es la unidad mínima de una imagen digital. En escala de grises de 8 bits, su valor va de **0** (negro) a **255** (blanco). Numéricamente, una imagen es una matriz bidimensional (escala de grises) o tridimensional (color).

Valor	Interpretación	Ejemplo visual
0	Negro	■
64	Gris oscuro	
128	Gris intermedio	
192	Gris claro	
255	Blanco	□

Código: matriz 3×3

```
import numpy as np

imagen = np.array([
    [0,   50, 100],
    [150, 200, 255],
    [255, 100, 0]
], dtype=np.uint8)

print(imagen.shape)    # (3, 3)
print(imagen[0, 0])    # 0 → esquina sup. izq.
print(imagen[1, 2])    # 255 → fila 1, col. 2
```

Explicación

- **dtype=np.uint8**: enteros sin signo de 8 bits (0–255)
- **shape**: devuelve (alto, ancho) o (alto, ancho, canales)
- **imagen[1, 2]**: fila 1, columna 2 (índices desde cero)
- **Origen (0,0)**: esquina superior izquierda

Made with GAMMA

BLOQUE 2 · CANALES DE COLOR

Modelos de color: RGB, BGR y HSV

RGB

Rojo, Verde, Azul. Modelo aditivo estándar. Cada píxel es una terna (R, G, B) de 0 a 255. Blanco = (255, 255, 255).

BGR (OpenCV)

OpenCV almacena en BGR por defecto. Sin conversión, los colores aparecen invertidos. Usar `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

HSV

Tono (0–179), Saturación (0–255), Valor (0–255). Ideal para segmentar colores. Conversión: `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`.

Color	RGB	BGR	HSV (H)
Negro	(0, 0, 0)	(0, 0, 0)	(0, 0, 0)
Blanco	(255, 255, 255)	(255, 255, 255)	(0, 0, 255)
Rojo	(255, 0, 0)	(0, 0, 255)	(0, 255, 255)
Verde	(0, 255, 0)	(0, 255, 0)	(60, 255, 255)
Azul	(0, 0, 255)	(255, 0, 0)	(120, 255, 255)

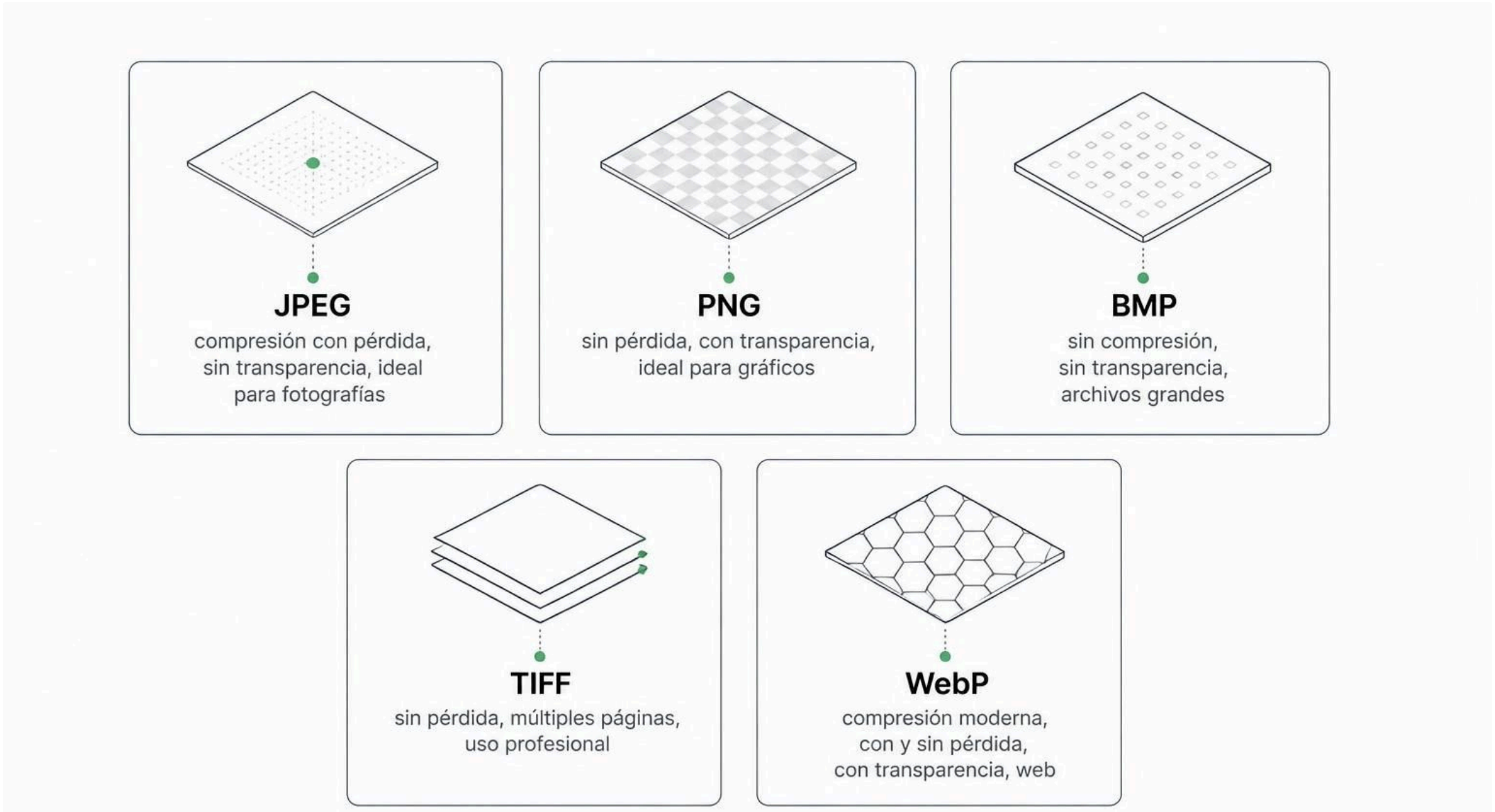
Made with GAMMA

BLOQUE 3 · RESOLUCIÓN Y FORMATOS

Resolución, dimensiones y formatos de imagen

Las dimensiones se expresan como **ancho × alto** en píxeles. Una imagen 640×480 tiene 307.200 píxeles. En color (3 canales), ocupa aproximadamente **921.600 bytes** en memoria sin compresión.

Resolución	Píxeles totales	Grises (KB)	Color BGR (KB)	Uso típico
320 × 240	76.800	75	225	Dispositivos embebidos
640 × 480	307.200	300	900	VGA estándar
1280 × 720	921.600	900	2.700	HD (720p)
1920 × 1080	2.073.600	2.025	6.075	Full HD (1080p)



El tamaño del archivo puede ser menor que el de la matriz en memoria gracias a la compresión. El formato PNG es ideal para resultados intermedios; JPEG para fotografías finales.

Made with GAMMA

BLOQUES 4 Y 5 · OPENCV

OpenCV: instalación, carga y visualización

OpenCV (Open Source Computer Vision Library) es la biblioteca de referencia para procesamiento de imágenes en Python. Trabaja con matrices NumPy y ofrece funciones para cargar, transformar, analizar y guardar imágenes.

Instalación en VS Code

```
python -m venv env

# PowerShell:
.\env\Scripts\Activate.ps1

python -m pip install --upgrade pip
python -m pip install opencv-python numpy
```

Verificar instalación:

```
import cv2, numpy as np
print(cv2.__version__)
print(np.__version__)
```

Cargar, mostrar y guardar

```
import cv2

# Cargar (devuelve None si falla)
img = cv2.imread("imagenes/foto.jpg",
                 cv2.IMREAD_COLOR)

if img is not None:
    cv2.imshow("Original", img)
    cv2.waitKey(0)          # Espera tecla
    cv2.destroyAllWindows()  # Cierra ventanas

    # Guardar resultado
    cv2.imwrite(
        "resultados/salida.png", img)
else:
    print("Error: no se cargó la imagen")
```

- **cv2.waitKey(0)**: espera indefinidamente una tecla
- **cv2.destroyAllWindows()**: cierra todas las ventanas
- **cv2.imwrite()**: el formato depende de la extensión

Made with GAMMA



BLOQUES 6 Y 7 · BRILLO Y CONTRASTE

Brillo, contraste e histogramas

Brillo (suma/resta)

Sumar aumenta el brillo; restar lo reduce. Usar `np.clip()` para evitar desbordamiento en uint8:

```
mas_clara = np.clip(
    img.astype(np.int16) + 40,
    0, 255).astype(np.uint8)
```

cv2.convertScaleAbs()

Alternativa práctica: `alpha` (multiplicador) y `beta` (desplazamiento):

```
ajustada = cv2.convertScaleAbs(
    img, alpha=1.2, beta=30)
```

Contraste (multiplicación)

Factor > 1 aumenta contraste; < 1 lo reduce. Puede saturar zonas extremas:

```
contraste = np.clip(
    img.astype(np.float32) * 1.5,
    0, 255).astype(np.uint8)
```

Histograma con cv2.calcHist()

Muestra la distribución de intensidades (no conserva posición espacial):

```
gris = cv2.cvtColor(
 img, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist(
 [gris], [0], None,
 [256], [0, 256])
```

Made with GAMMA

BLOQUE 8 · REDIMENSIONAMIENTO

Redimensionamiento e interpolación

Al cambiar el tamaño de una imagen, OpenCV calcula nuevos píxeles mediante **interpolación**. El método elegido afecta la calidad visual del resultado.

cv2.resize() — ejemplos

```
import cv2

# Dimensiones fijas (ancho, alto)
reducida = cv2.resize(
    img, (320, 240),
    interpolation=cv2.INTER_AREA)

# Escala relativa (fx, fy)
ampliada = cv2.resize(
    img, None,
    fx=1.5, fy=1.5,
    interpolation=cv2.INTER_CUBIC)
```

⚠ **¡Atención!** `dsize` usa (ancho, alto), pero `shape` devuelve (alto, ancho). Invertirlos es un error frecuente.

Métodos de interpolación

INTER_NEAREST

Vecino más cercano. Rápido, bordes escalonados.

INTER_LINEAR

Bilineal. Opción general equilibrada.

INTER_CUBIC

Bicúbica. Resultados suaves al ampliar.

INTER_AREA

Por área. Recomendada para reducir.

Made with GAMMA

BLOQUE 9 · RECORTE Y ROI

Recorte y regiones de interés (ROI)

Una **región de interés (ROI)** es un área rectangular seleccionada de la imagen para procesarla de forma independiente. Se obtiene mediante **slicing** de NumPy: `imagen[y1:y2, x1:x2]`.

Sintaxis de slicing

```
region = imagen[50:200, 100:300]
# filas 50→199, columnas 100→299

alto_roi = region.shape[0]
ancho_roi = region.shape[1]
print(f"ROI: {ancho_roi}x{alto_roi}")
```

El límite superior es **exclusivo**: `50:200` incluye 150 filas (de la 50 a la 199).

Ejemplo completo con validación

```
import cv2

img = cv2.imread("imagenes/foto.jpg")

if img is not None:
    roi = img[50:200, 100:300]

    if roi.size > 0:
        cv2.imshow("ROI", roi)
        cv2.imwrite(
            "resultados/recorte.png", roi)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
    else:
        print("ROI vacía: coordenadas fuera de límites")
else:
    print("No se cargó la imagen")
```

Si las coordenadas superan los límites, el recorte puede quedar vacío o tener dimensiones inesperadas. Validar siempre con `region.size > 0`.

Made with GAMMA

BLOQUE 10 · LABORATORIO INTEGRADOR

Laboratorio 1: Análisis y manipulación con OpenCV

Construye un programa que cargue una fotografía, analice sus propiedades y genere versiones modificadas. Divide el trabajo en fases progresivas.

01

Fase 1 — Carga y validación

Cargar con `cv2.imread()` y verificar con `if img is not None`.

02

Fase 2 — Propiedades

Consultar `shape`, `size` y `dtype`. Calcular canales: `shape[2]` si existe.

03

Fases 3–5 — Transformaciones

Escala de grises (`cv2.cvtColor`), brillo (`convertScaleAbs`) y contraste (multiplicación con `np.clip`).

04

Fases 6–8 — Análisis y geometría

Histograma (`cv2.calcHist`), redimensionar (`cv2.resize`) y recortar ROI con slicing.

05

Fase 9 — Guardar resultados

Guardar al menos tres versiones en `resultados/` con `cv2.imwrite()`. Cerrar ventanas.

Errores frecuentes: No verificar `None` tras `imread()`, confundir BGR con RGB, invertir ancho/alto en `resize()`, olvidar `cv2.waitKey()` y guardar en carpetas inexistentes.

Made with GAMMA